サンプル問題

Ｐｙｔｈｏｎプログラミング能力認定試験

１　級

正答・解説

サーティファイ
情報処理能力認定委員会

＜解説＞

問 1

(1)	(2)	(3)	(4)	(5)	(6)
ア	イ	ア	イ	イ	ア

(2) メソッドの定義時に初期値を設定しておき，実行時に引数を省略した場合にその初期値が代入される引数は，デフォルト引数である。

(4) ラムダ式で記述できる式は 1 行だけであり，複数行記述することはできない。

(5) Python のプログラムが記述されたファイルをモジュールといい，そこに記述されたクラスや関数をインポートするには「`from` モジュール名（拡張子を除くファイル名） `import` クラス名（または関数名）」の形で記述する。したがって，「`from ProgA import ClassA`」と記述する必要がある。

問２

(7)	(8)	(9)	(10)	(11)	(12)	(13)	(14)
イ	ア	ア	ウ	イ	ウ	ア	イ

(7)(8) `__init__` メソッドが実行されるのは，インスタンスの生成時である。なお，`__init__` メソッドのことをコンストラクタと呼ぶこともある。

(9)(10) `calc_bmi` メソッドはインスタンスメソッドであり，第 1 引数はどのような場合も省略することはできない。なお，第 1 引数の変数名は決められていないが，`self` が慣習的に使われている。

(11) 継承元の `BMI` クラスに加えて新たに追加された変数は `boundary_age` と `age` の二つである。その他の `message` や `rohrer` といった変数は，メソッド内のみで使用されるローカル変数である。

(12) 継承元の `BMI` クラスに加えて新たに追加されたメソッドは，`calc_rohrer`，`calc_index`，`which_index` の三つである。`__init__` メソッドは継承元の `BMI` クラスに定義されており，それをオーバーライドしている。

(13) クラスメソッドは，インスタンスからもクラスからも実行できるメソッドであるが，スタティックメソッドと異なり，クラス内の変数を参照することが可能である。また，インスタンスを生成しなくても実行できる。

(14) クラス変数は，インスタンスからもクラスからも参照することのできる変数であり，インスタンスを生成しているかどうかは問わない。

問３

(15)	(16)	(17)	(18)	(19)
ウ	エ	イ	ウ	ウ

(15) 解答群にある「&」と「|」はそれぞれビットごとの論理積と論理和を求める演算子であり，「<<」と「>>」はそれぞれ左シフトと右シフトを行う演算子である。ビット単位の論理演算とシフト演算では，シフト演算の方が優先的に処理されることに注意して解答群の式の計算をそれぞれ行うと，ア：0，イ：4，ウ：7，エ：6 である。したがって，「a | b >> 1」となる。

(16) 初期値 p = 1 に 1 から 5 までの整数を順に掛けながら list_a に格納していくので，3〜5 行目の繰返し処理が終わった時点での list_a には[1, 2, 6, 24, 120]が格納されている。解答群はどれも list_a のスライスとリスト[24, 120]の結合を行う処理であるから，出力されたリストから[24, 120]を除いた[2, 6, 24]が list_a のスライスとなるようなものを選べばよい。[2, 6, 24]は list_a のインデックス 1 から 3 までと同じである。したがって，「list_a[1:4]」となる。

(17) while の繰返し処理の 1 回目では，4 行目の処理により d の値は 50 となり，5，6 行目が実行される。解答群のうち，break は繰返し処理からの離脱であり，list_a が空のまま 12 行目が実行されるので不適切である。また，return は関数内でのみ使用できるので，これも不適切である。一方，continue は繰返し処理をそこで中止し，繰返しの次の処理を継続する。この場合，5 行目の条件分岐で d の値が 10 以下になるまで繰り返されることになる。d の値は 4 行目で 50，25，12，6，3，1 と変化し，0 で while の継続条件が False となり，処理が終わる。このうち，7 行目以降に進めるのは，6，3，1 の場合のみである。7〜11 行目の処理では 1 以上 d 未満の数値のうち 4 未満のものを list_a に追加しているので，list_a の値は[1, 2, 3, 1, 2]となる。問題文の出力結果と一致する。したがって，「continue」となる。なお，pass は何も処理をしない文であり，if 文の条件に関係なくそのまま 7 行目以降が処理される。すなわち，7〜11 行目の処理が d = 50，25，12，6，3，1 に対して実行されるため，list_a の値は[1，2，3，1，2，3，……，1，2]となり，不適切である。

（18）関数 func の外で定義した 1，2 行目の list_a，list_b の値は，それぞれ[1，2，3，4]および[1，2]である。一方，関数 func 内で定義した list_a，list_b の値は，それぞれ[2，4，6，8]および[2，4]である。9 行目の関数 func の実行後に list_c の値が[2，4，1，2，3，4]となったのであるから，関数 func 内の list_b と関数の外の list_a が結合されていることがわかる。つまり，list_b のみ関数の内外で共通して使用している。したがって，「global list_b」となる。

（19）id 関数は，オブジェクト ID を取得する関数である。オブジェクトとは，数値や文字列，インスタンスなどの Python で扱うデータのことであり，各データの生成時に固有の ID が付与される。イは list_b の値が[0，2]となり，リストのサイズが異なるため，不適切である。ア，ウ，エをそれぞれ空欄に入れると，list_b の値はいずれも[0，2，4]となるが，ア，エは新たにオブジェクトを生成しているので，list_b のオブジェクト ID は list_a のオブジェクト ID と異なる値となる。これにより，id1 == id2 の結果は False となる。一方，ウの list_b = list_a では，list_b に list_a のオブジェクトを代入している（オブジェクトを新しく生成しているわけではない）ので，5〜7 行目のように要素の値を変更してもオブジェクト ID は変わらない。これにより，id1 == id2 の結果は True となる。したがって，「list_a」となる。

問４

(20)	(21)	(22)	(23)	(24)
ア	ア	ウ	イ	イ

(20) round 関数は，与えられた引数に最も近い整数を返す関数である（偶数丸め）。17 / 3 の値は 5.6666……であるから，c の値は 6 となる。また，divmod 関数は，divmod(a, b)の形で，a を b で割ったときの商と剰余をタプルとして返す関数である。したがって，d と m にはそれぞれ商の 5 と剰余の 2 が代入される。pow 関数は pow(d, m)の形で，d の m 乗を返す関数であるため，e には 5 の 2 乗である 25 が代入され，25 - 6 = 19 となる。したがって，「19」が表示される。

(21) set 型に対する演算子「&」は，積集合（2 つの集合に共通する要素のみ持つ集合）を求める演算子である。koku_su と su_sha の集合はそれぞれ{'浩二', '和也'}および{'浩二', '美香'}となる。「^」は対称差集合（2 つの集合のどちらか片方にしかない要素のみ持つ集合）を求める演算子であり，和集合（2 つの集合の要素を持つ集合）から積集合を除いたものである。koku_su と su_sha の和集合と積集合はそれぞれ{'浩二', '和也', '美香'}，{'浩二'}である。したがって，「{'和也', '美香'}」が表示される。なお，set 型の要素は順序を持たないため，「{'美香', '和也'}」と表示される場合もある。

(22) 2 行目はアンパック代入と呼ばれ，リストやタプルなどの各要素をそれぞれ別の変数に同時に代入する機能である。リスト u の要素数は 6，アンパック後の変数は 4 つあるが，変数に必ず値を入れなければならない。ここで，「*」（アスタリスク）のついた変数を用意すると，可変長のリストとして値を代入することができる。u1 には 1 行目のリスト u の 1 つ目の要素が代入され，u3，u4 にはリスト u の最後から 2 番目の要素と最後の要素が代入される。残りの要素が「*」のついた u2 にまとめてリストとして代入される。したがって，「[4, 6, 8]」が表示される。

(23) yield で値を返す gen 関数はジェネレータであり，0 から k 未満の整数に 2 を掛けて返すイテレータを生成する。6 ～ 8 行目の next 関数でイテレータの 0，2，4 までを読み込んでおり，10 行目以降読み込まれるのは 6 と 8 の 2 つであるため，sum にはその和である 14 が格納される。したがって，「14」が表示される。

(24) try，except，finally は，例外が発生した際の処理を定義するものである。try 句以下の処理で発生する例外のうち，except 句に想定される例外の種類を記述し，対応した例外ごとの処理を定義する。そして，最終的に行う処理を finally 句で定義する。5 行目の処理では，b = 456.0 を str 関数で文字列に変換し，a + str(b)の値が「123456.0」となる。[10]は文字列の先頭から 11 番目の文字を取り出すための記述であるが，「123456.0」の長さは 8 であり，あてはまる要素（Index）がないため，例外 IndexError が発生する。また，c には 3 行目で代入し

た None が格納されているので，「Index Error!」の後に改行が入り，13 行目の処理が行われる。したがって，「Index Error!」，「None」の順に表示される。

問５

(25)	(26)	(27)	(28)	(29)
イ	ウ	イ	ア	イ

(25) dict_a['Grape']は，辞書 dict_a に格納されている要素のうち，キーが'Grape'である要素の値を指す。したがって，「6」が表示される。

(26) 6 行目でキーが'Papaya'，値が 3 の要素が辞書 dict_a に追加されている。なお，辞書は挿入順が保たれるため，辞書の最後に挿入される。そして，8 行目では 1 行目で定義した dict_a['Apple']と 6 行目新たに追加した dict_a['Papaya']の値の積を求めている。したがって，「24」が表示される。

(27) 10～14 行目で，dict_a のキーをキーとする辞書 dict_b が生成されている。その値は，i = -4 から始まり繰返し処理ごとに 2 増加する i に対して，i ** 2 – 4 の演算結果が格納されている。つまり，dict_b の値は{'Apple': 12, 'Banana': 0, 'Grape': -4, 'Persimmon': 0, 'Papaya': 12}である。したがって，「0」が表示される。

(28) 20 行目の zip 関数は，複数のイテラブルな対象に対し，平行に反復処理をして，処理ごとのタプルを生成する。zip(list_b, list_c)で，list_b と list_c の各要素番号別のタプルのイテレータが生成される。このイテレータを元に dict 関数で辞書 dict_c を生成する。よって，dict_c の値は{'Blueberry': 0, 'Cherry': 1, 'Grapefruit': 3, 'Lemon': 4, 'Plum': 6}となる。したがって，「1」が表示される。

(29) 23 行目の演算子「|」は，辞書型のデータを結合する演算子であるので，辞書 dict_d は {'Apple': 8, 'Banana': 2, 'Grape': 6, 'Persimmon': 9, 'Papaya': 3, 'Blueberry': 0, 'Cherry': 1, 'Grapefruit': 3, 'Lemon': 4, 'Plum': 6}となる。なお，キーに重複がある場合は，演算子の右側のデータが優先される。

26～28 行目では，dict_d の各キーが'P'から始まるかどうかを startswith('P')で調べ，True であればそのキーに対応する値を代入する処理となっている。したがって，「18」が表示される。

問６

(30)	(31)	(32)	(33)	(34)
エ	イ	ア	ア	イ

(30) recipes のキー（料理名）のうち，料理名 recipe_name に一致するキーがあるかどうかを判断し，無い場合に 10 行目を実行する必要がある。したがって，「recipe_name not in recipes」となる。なお，解答群の is は同一オブジェクトであるかどうかを判断する演算子である。

(31) 必要な材料が在庫にあるかを算出する処理である。必要な材料のキー（材料名）が在庫のキー（材料名）にすべて存在する場合，必要な材料がすべて在庫にある状態と言える。これを算出するためには，キーを対象に set 型の差集合を求める演算を行えばよい。在庫のセットを必要な材料のセットから引くことで足りない材料を求められる。したがって，「set(zairyou) - set(stocks)」となる。なお，ウやエのように，演算子「-」を使って辞書型同士で直接差分を求めることはできない。

(32) 材料のキーが在庫のキーにすべて存在する場合の処理である。変数 fu に不足する材料の数量を代入し，32 行目で fu が正の数であれば辞書 fusoku に不足分の材料の要素を追加している。つまり，31 行目では，必要な材料の数量から在庫分を差し引けばよいが，分量（人数）を考慮する必要がある。したがって，「ninzu * suryou - stocks[zai]」となる。

(33) 数量が不足する材料を不足数量とともに表示する処理である。辞書 fusoku のキーは材料名，値は不足分の数量であり，それぞれを zai と fu に代入しながら繰返し処理を行う。したがって，「zai, fu in fusoku.items()」となる。イの keys はキーのみ，エの values は値のみを取り出すメソッドのため不適。ウの popitem は要素を取り出すメソッドであるが，繰返し処理のイテラブルオブジェクトとしては使用できない。

(34) suggest は，在庫だけで作れる料理を表示する関数で，judge 関数の引数 show_full を False にして，材料が全く無い料理を表示させないようにすればよい。したがって，「judge(ninzu, recipe, False)」となる。

問 7

(35)	(36)	(37)	(38)	(39)
ウ	イ	イ	エ	イ

(35) 15 行目の set メソッドは，グラフの軸に関する情報を設定するメソッドである。引数に辞書型の可変長引数をとり，直前の行で定義した辞書型の変数 kwargs を引数として渡す処理である。辞書型の変数を引数として渡すためには，アスタリスク二つ「**」を変数の前につければよい。したがって，「**kwargs」となる。なお，タプルを可変長引数として渡す場合には，アスタリスク一つ「*」を変数の前につける。

(36) バブルソートで値の交換を行う処理である。要素番号 j の要素のほうが右隣の要素よりも大きければ交換するので，空欄にあてはまるのは「li[j], li[j + 1] = li[j + 1], li[j]」である。なお，これはタプルを用いた値の交換である。

(37) _proc_times に計測した処理時間をミリ秒単位に変換して追加する処理である。perf_counter メソッドが返す値は秒単位なので，ミリ秒単位にするために 1000 倍する必要がある。したがって，「self._proc_times.append((end - strart) * 1000)」となる。

(38) numpy ライブラリの median 関数を使って処理時間の中央値を求め，出力する処理である。numpy ライブラリはプログラム冒頭で「import numpy as np」とインポートされている。median 関数を使用するには np.median とする必要がある。また実行結果を見ると，中央値の数値は小数第 3 位まで表示されているため，固定小数点表現の「f」を使用する。したがって，「np.median(self._proc_times):.3f」となる。

(39) BubbleSort クラスからインスタンスを生成する処理である。7 行目のコンストラクタの引数には，self と num_list が指定されているが，self はインスタンス自身であり，インスタンス生成時には書く必要がない。num_list はインスタンス生成時に必ず指定する必要があり，そのためのリストは 48 行目で生成されている。したがって，「BubbleSort(num_list)」となる。

〔 メ モ 用 紙 〕

試験問題は著作権法上の保護を受けています。
試験問題の一部または全部について，サーティファイから文書による許諾を得ずに，いかなる方法においても私的使用の範囲を超えて，無断で複写，複製することを禁じます。
無断複製，転載は損害賠償，著作権法の罰則の対象になることがあります。
©CERTIFY Inc.2024